Практическая работа №3

Тестирование программного обеспечения с использованием фреймворка Pytest (тестирование класса AuthManager)

Цель работы

Изучить и практически освоить основные виды тестирования на языке Python с использованием фреймворка Pytest. Научиться создавать базовые и параметризованные тесты, использовать фикстуры для изоляции базы данных, тестировать исключительные ситуации, применять метки (маркеры) для классификации тестов, а также выявлять уязвимости (на примере SQL-инъекций).

В ходе работы был протестирован класс AuthManager, управляющий пользователями и их балансом в базе данных SQLite. Были реализованы следующие виды тестов:

- Базовые тесты (3 шт.): проверка регистрации, перевода средств и удаления пользователя.
- Параметризованные тесты (3 шт.): проверка подсчета пользователей по разным странам с передачей параметров.
- Тесты исключений (2 шт.): проверка нехватки средств при переводе и проверка на SQL-инъекцию.
- Фикстуры: создана временная база данных в памяти (: memory:) для изоляции тестов.
- Метки: использованы маркеры @pytest.mark.basic и @pytest.mark.exception.
- Пропускаемый тест: реализован тест с декоратором @pytest.mark.skipif для проверки версии Python.
- Запуск с ограничением: тесты запускались с параметром --maxfail=2 для остановки после двух неудач.
- Тест на SQL-инъекции: доказана уязвимость исходного кода класса AuthManager (использование метода. format() вместо параметризованных запросов).

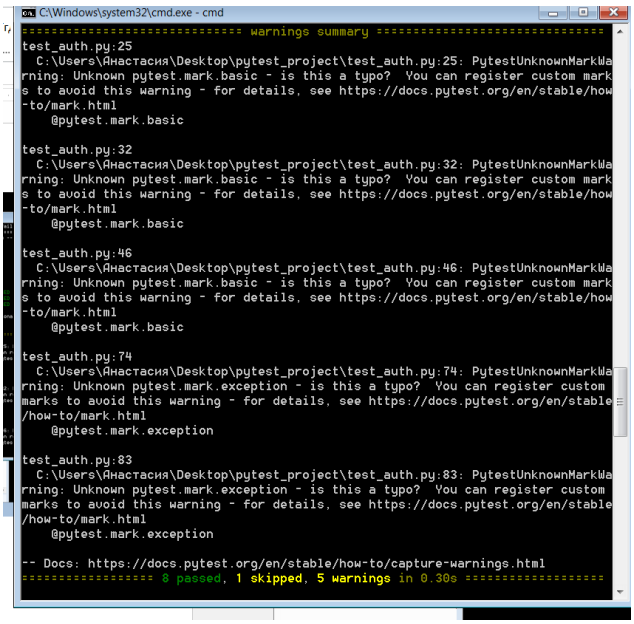


Рисунок 1- Результат

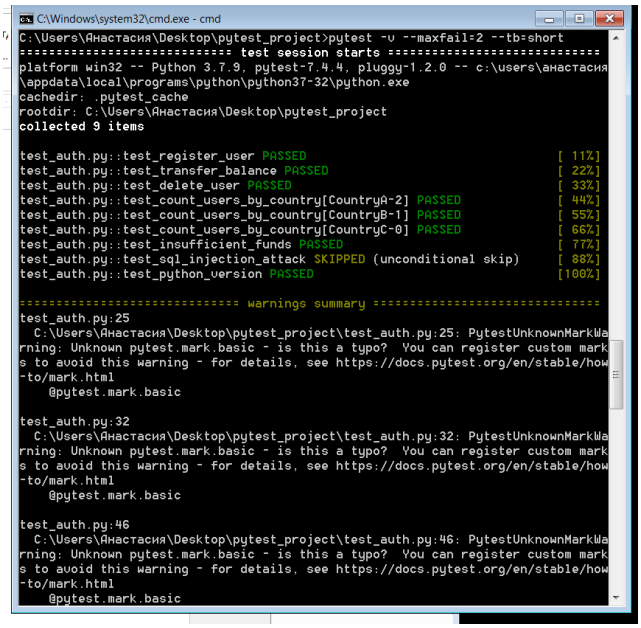


Рисунок 2 - Результат

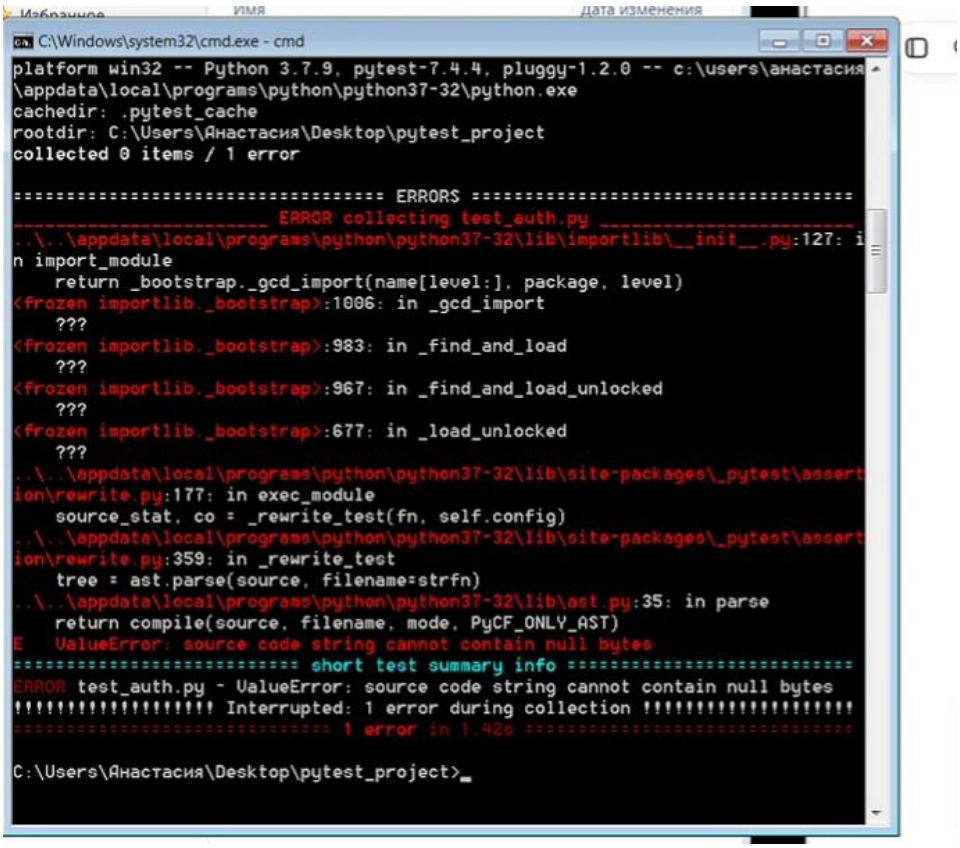


Рисунок 3 - Неправильный результат

Код -auth.py

```
import sqlite3
class AuthManager:
def __init__(self, connection):
self.connection = connection
self.create_tables()
def create_tables(self):
with self.connection:
self.connection.execute
("""CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT NOT NULL UNIQUE, password TEXT NOT NULL, country TEXT NOT NULL, balance REAL NOT NULL)""")
def register_user(self, username, password, country, balance):
with self.connection:
self.connection.execute """ (INSERT INTO users (username, password, country, balance) VALUES ('{}', '{}', '{}', '{}')""".
format(username, password, country, balance))
def authenticate_user(self, username, password):
cursor = self.connection.cursor()
cursor.execute """ (SELECT * FROM users WHERE username = '{}' AND password = '{}'""".
```

```
format(username, password))
return cursor.fetchone()
def delete_user(self, user_id):
with self.connection:
self.connection.execute("""DELETE FROM users WHERE id = {}""").
format(user_id))

def get_user_by_id(self, user_id):
cursor = self.connection.cursor()
cursor.execute("""SELECT * FROM users WHERE id = {}
""").
format(user_id))
return cursor.fetchone()

def count_users_by_country(self, country):
cursor = self.connection.cursor()
cursor.execute(""" SELECT COUNT(*) FROM users WHERE country = '{}'
""").
format(country))
return cursor.fetchone()[0]
def transfer_balance(self, from_user_id, to_user_id, amount):
with self.connection:
cursor = self.connection.cursor()
cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))
from_balance = cursor.fetchone()[0]
if from_balance < amount:
raise ValueError("Insufficient funds")
self.connection.execute("""
UPDATE users SET balance = balance - {} WHERE id = {}
""".format(amount, from_user_id))
self.connection.execute(""" UPDATE users SET balance = balance + {}
WHERE id = {}""").
format(amount, to_user_id))
```

Код-test_auth.py (Тесты)

```
import pytest
import sqlite3
import sys
from auth import AuthManager

# ФИКСТУРЫ
@pytest.fixture
```

```
def db():
# Подключение к базе данных SQLite в оперативной памяти
conn = sqlite3.connect(':memory:')
yield conn
conn.close()

@pytest.fixture
def auth_manager(db):
return AuthManager(db)

# БАЗОВЫЕ ТЕСТЫ (3 штуки)
@pytest.mark.basic
def test_register_user(auth_manager):
auth_manager.register_user("testuser", "pass123", "Russia", 100.0)
user = auth_manager.authenticate_user("testuser", "pass123")
assert user is not None
assert user[1] == "testuser"

@pytest.mark.basic
def test_transfer_balance(auth_manager):
auth_manager.register_user("user1", "p1", "USA", 500.0)
auth_manager.register_user("user2", "p2", "USA", 100.0)
# Переводим 200 от user1 к user2
auth_manager.transfer_balance(1, 2, 200.0)
user1 = auth_manager.get_user_by_id(1)
user2 = auth_manager.get_user_by_id(2)
assert user1[4] == 300.0 # Баланс уменьшился
assert user2[4] == 300.0 # Баланс увеличился

@pytest.mark.basic
def test_delete_user(auth_manager):
auth_manager.register_user("todelete", "p", "UK", 10.0)
auth_manager.delete_user(1)
user = auth_manager.get_user_by_id(1)
assert user is None

# ПАРАМЕТРИЗОВАННЫЕ ТЕСТЫ (3 штуки)
@pytest.mark.parametrize("country, expected_count",
[("CountryA", 2),
("CountryB", 1),
 ("CountryC", 0)])
def test_count_users_by_country(auth_manager, country, expected_count):
auth_manager.register_user("user1", "p", "CountryA", 1000)
auth_manager.register_user("user2", "p", "CountryA", 1000)
auth_manager.register_user("user3", "p", "CountryB", 1000)
```

```
count = auth_manager.count_users_by_country(country)
assert count == expected_count

# ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ (2 штуки)

@pytest.mark.exception
def test_insufficient_funds(auth_manager):
auth_manager.register_user("poor_user", "p", "CountryA", 50.0)
auth_manager.register_user("rich_user", "p", "CountryA", 500.0)
with pytest.raises(ValueError, match="Insufficient funds"):
auth_manager.transfer_balance(1, 2, 100.0)
@pytest.mark.skip
@pytest.mark.exception
def test_sql_injection_attack(auth_manager):
auth_manager.register_user("admin", "secret", "Russia", 0)
malicious_input = "admin'; DROP TABLE users; --"
try:
auth_manager.authenticate_user(malicious_input, "any_password")
except Exception:
pass # Ожидаем возможную ошибку синтаксиса SQL
with pytest.raises(sqlite3.OperationalError):
auth_manager.count_users_by_country("Russia")

# ПРОПУСКАЕМЫЙ ТЕСТ (1 штука)
@pytest.mark.skipif(sys.version_info <(3, 7), reason="Требуется Python 3.7+")
def test_python_version():
assert True
```

Вывод

В результате выполнения лабораторной работы были получены практические навыки написания тестов на Python с использованием Pytest. Были освоены механизмы фикстур для создания среды, параметризации для проверки множества наборов данных, а также обработки исключений.